

Introduction to SQL

- The learning objectives for this week are:
 - Knowing how to use **SQL Server Management Studio** to connect to the SQL Server RDBMS
 - Knowing how to use SQL Server Management Studio to execute database queries
 - Knowing how to create database tables using SQL
 - Knowing how to insert data into a table using SQL
 - Knowing how to retrieve data from the database using SQL

SQL

- **Structured Query Language** (SQL) is the standard database language for relational databases
- With SQL we can, create the database and table structures, manipulate data in the tables (insert, update, delete) and retrieve data from the tables
- An **SQL query** is a single statement in which describes what we want from the database
- The query operates on **tables** and builds a result table from one or more tables in the database

```
SELECT teacher_number, first_name, surname, phone  
FROM Teacher
```

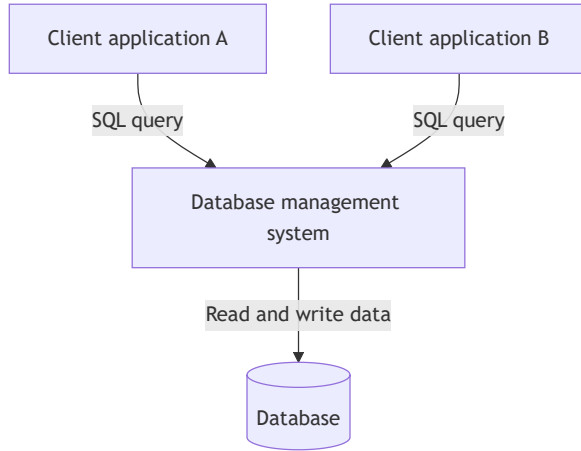
teacher_number	first_name	surname	phone
h180	Seppo	Kokki	09-808800
h290	Sisko	Saari	09-776655
...	...	...	...

SQL syntax variations

- There is an official standard for SQL maintained by ISO/IEC and **most of the syntax is the same in all RDBMS**, but **every major RDBMS introduces its own extensions, syntax tweaks, or behaviors**
- However, learning the SQL syntax of a specific RDBMS makes it very easy to learn the minor differences of another one

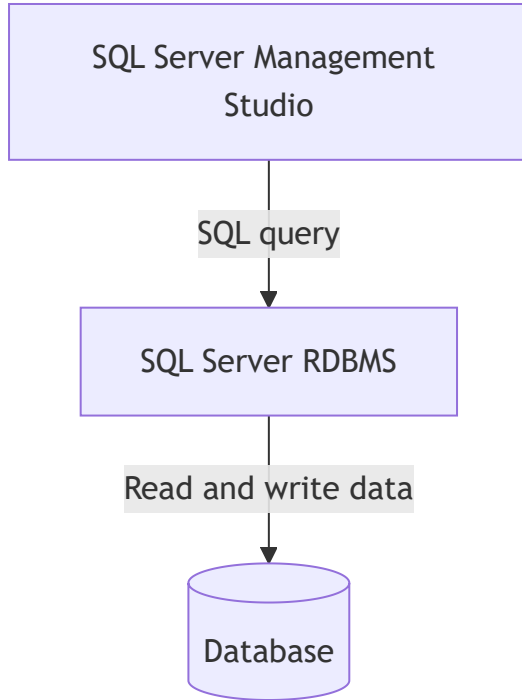
Operation	SQL Server	PostgreSQL
Limit rows	<code>SELECT TOP 5 * FROM table</code>	<code>SELECT * FROM table LIMIT 5</code>
Current datetime	<code>GETDATE()</code>	<code>NOW()</code>
String length	<code>LEN(string)</code>	<code>LENGTH(string)</code>

Communicating with a RDBMS using SQL



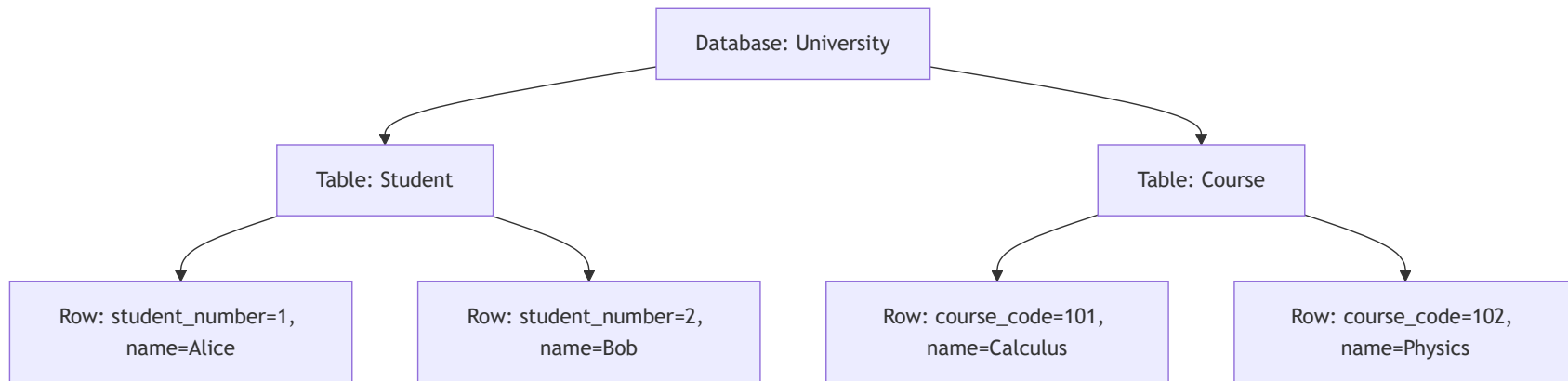
- SQL is used with a relational database management systems (RDBMS), such as **Microsoft SQL Server**, which we will be using during the course
- RDBMS software can be running on local computer on a server on the internet
- We can send database queries to a RDBMS using a **client application**, which can be, for example a programming interface, a command line interface or a graphical interface
- During the course we have the option to either install the SQL Server on our computer or use the SQL server running in Haaga-Helia's servers

Communicating with a RDBMS



- During this course we will be using a graphical interface called **SQL Server Management Studio** to communicate with the SQL Server RDBMS
- With SQL Server Management Studio we can for example inspect and manage database related information, perform database queries and visualize the structure of the database tables

Creating a database



- In SQL, a **database** is a named collection of **tables**
- In addition to tables, database holds different kinds of configuration, for example related to access control
- We can create a database with the `CREATE DATABASE` statement

```
CREATE DATABASE University
```

Creating a table

- The actual data of a database lives inside **tables**
- Table has a name and a collection of **columns**
- We can create a table with the `CREATE TABLE` statement

```
CREATE TABLE Student (  
    student_number INTEGER,  
    first_name VARCHAR(50),  
    surname VARCHAR(50)  
)
```

student_number	first_name	surname
1001	Alice	Johnson
1002	Bob	Smith
1003	Charlie	Nguyen

Creating a table

- Table and column names **should describe the information they store**
 - The “Student” table contains rows that represent students
 - The “first_name” column contains the first name of the student
- Table and column names should consist of letters, digits or underscores. They **should not contain whitespace**
- In column names, a underscore symbol () is commonly used instead of whitespace. For example “first_name” instead of “first name”
- Table names are commonly in **singular format**, for example “Student”
- Each column has a **data-type** that determines the kind of values the column can have
- For example an `INTEGER` type of column can only contain integer values and `VARCHAR(n)` column string values

SQL Server data types

Category	Data type	Description	Example value
Integer Types	INTEGER	Integer from -2,147,483,648 to 2,147,483,647	42
	BIGINT	Integer from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	90000000000
Decimal Types	DECIMAL(p,s)	Fixed precision and scale numeric data (decimal numbers)	123.45
Character Types	CHAR(n)	Fixed-length non-Unicode string (max 8,000 chars)	'ABC'
	VARCHAR(n)	Variable-length non-Unicode string	'Hello!'
Date & Time	DATE	Date with year, month and day (YYYY-MM-DD)	'2026-08-28'
	TIME	Time with hours, minutes and seconds (HH:MM:SS)	'14:30:00'

Constraints

- **Constraints** specify rules for the data in a table
- For example `NOT NULL` constraint ensures that a column cannot have a `NULL` (empty) value
- The `NOT NULL` constraint is defined **after the column type** in the `CREATE TABLE` statement

```
CREATE TABLE Student (  
    student_number INTEGER NOT NULL,  
    first_name VARCHAR(50) NOT NULL,  
    surname VARCHAR(50) NOT NULL  
)
```

Primary key constraint

- **Primary key** uniquely identifies each row in the table
- **Primary key constraint** prevents duplicate rows to exist for the table
- Primary key constraint is defined with the `PRIMARY KEY` constraint **after the column definitions** in the `CREATE TABLE` statement

```
CREATE TABLE Student (  
    student_number INTEGER NOT NULL,  
    first_name VARCHAR(50) NOT NULL,  
    surname VARCHAR(50) NOT NULL,  
  
    -- the primary key is the student_number column  
    CONSTRAINT Pk_Student PRIMARY KEY (student_number)  
)
```

Foreign key constraint

- **Foreign key** is a column or group columns whose values are required to match those of the primary key of the referenced table
- **Foreign key constraint** prevents foreign key not being matched by a primary key in the referenced table
- Foreign key constraint is defined with the `FOREIGN KEY` constraint **after the column definitions** in the `CREATE TABLE` statement

```
CREATE TABLE Laptop (  
    serial_number VARCHAR(10) NOT NULL,  
    student_number INTEGER NOT NULL,  
    -- the primary key is the serial_number column  
    CONSTRAINT Pk_Laptop PRIMARY KEY (serial_number),  
    -- the foreign key student_number references the student_number in the Student table  
    CONSTRAINT Fk_Student FOREIGN KEY (student_number)  
    REFERENCES Student(student_number)  
)
```

Inserting data

- We insert a new row into a table by defining the table name and the values for the columns
- A new row can be inserted with the `INSERT INTO` statement
- ⚠ String literals are defined with **single quotes**, for example `'Tampere'`

```
INSERT INTO Campus (campus_code, campus_name) VALUES ('c555', 'Tampere')
```

Constraint violations while inserting data

- Constraints are checked once a new row is inserted or updated
- If there are any constraint violations, **the new row won't be inserted**
- For example if `NOT NULL` constraint of a column is violated, there will be an error

```
-- ✖ campus_name column has a NOT NULL constraint, omitting it will cause an error  
INSERT INTO Campus (campus_code) VALUES ('c666')
```

Retrieving data

- The `SELECT` statement is used to **retrieve data from a table**
- It specifies the **columns** to return, the **table** to retrieve the data from, and optionally a **condition** to filter the rows
- The result is a **result table** containing the selected columns and the **matching rows** from the target table

```
SELECT course_code, course_name, credits
FROM Course
WHERE credits > 3
```

course_code	course_name	credits
a730	E-business	4
a800	Finnish 1	6

Summary

- We communicate with a **RDBMS server**, such as SQL Server using a **client application**, such as SQL Server Management Studio
- We can create database tables using the `CREATE TABLE` statement
- `PRIMARY KEY` constraint is used to define the table's primary key
- `FOREIGN KEY` constraint is used to define a foreign key referencing primary key column of another table
- `INSERT INTO` statement is used to insert a new row for the table
- `SELECT` statement is used to select rows from a table